A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle

Zehao Lin[1*], **Xixuan Hao**[1,2*], **Renyu Fu**[1,4], **Shaobo Cui**[3]
Kai Chen[1], **Chunyu Li**[1], **Zhiyu Li**[1†], **Feiyu Xiong**[1]
[1] MemTensor (Shanghai) Technology Co., Ltd.
[2] The Hong Kong University of Science and Technology (Guangzhou)
[3] Shanghai Jiao Tong University [4] Peking University
{linzh, fury, chenk, licy, lizy, xiongfy}@memtensor.cn,
xhao390@connect.hkust-gz.edu.cn, shaobo.cui@sjtu.edu.cn,
furenyu26@stu.pku.edu.cn

arXiv:2604.16548v5 [cs.CR] 30 Oct 2026

Abstract

The emergence of writable, cross-session persistent memory in LLM agents introduces a qualitatively different threat landscape from conventional input-centric security concerns, characterized by three properties: persistence, statefulness, and propagation. To systematically characterize this landscape, we propose a Memory Lifecycle Framework that organizes attacks, defenses, and their cross-phase dependencies along two axes: six lifecycle phases (WRITE, STORE, RETRIEVE, EXECUTE, SHARE & PROPAGATE, FORGET & ROLLBACK) and four security objectives (Integrity, Confidentiality, Availability, Governance). This analysis in turn exposes the need for explicit security requirements at the system level, motivating *Verifiable Memory Governance* (VMG), a framework of five architectural primitives for specifying and evaluating auditable, recoverable control over memory state within declared system and threat-model boundaries. Our analysis motivates integrating provenance, versioning, and policy-aware retention with retrieval and execution defenses to support auditable prevention, containment, and recovery across the lifecycle.

Author update, 30 October 2026. Related work and VMG clarifications are summarized in Appendix F.

1 Introduction

Large language models (LLMs) are increasingly serving as the foundation for interactive and autonomous agents (Zhang et al., 2025d; Wang et al., 2024). As their role expands from generating responses to supporting decisions, tool use, and user interactions, their security risks also extend beyond isolated outputs (Zhan et al., 2024; Zhang et al., 2025c; Greshake et al., 2023). In recent years, research on the security of LLM-based agents has traditionally centered on two parts: knowledge encoded parametrically in model weights (Carlini et al., 2021; Bourtoule et al., 2021; Maini et al., 2024) and prompt manipulation within single- or multi-turn interactions (Liu et al., 2024; Russinovich et al., 2025). As these agents evolve from stateless chatbots into autonomous systems equipped with persistent long-term memory, however, the focus of security concern is shifting from transient inputs to durable state. This transition qualitatively changes the landscape of security: attackers no longer merely seek to influence an immediate response, but to manipulate how the system *remembers, retrieves, plans, and acts* across future interactions.

Memory matters because distinctions preserved from past experience can improve later decisions. Lin et al. (2026b) distinguish the decision value available in a history, retained in persistent state, and realized by a bounded reader under specified demands and losses, while accounting for recovery and revision costs. For this survey, that perspective motivates protecting both authorized use and useful continuity: attacks can corrupt retained support or its use, while defenses can discard legitimate support. The design problem therefore connects utility, cost, and governance: what merits retaining, who may access or revise it, and how interventions affect future work.

Existing prompt-injection and RAG-corruption mitigations are fundamentally limited for Long-Term Memory (LTM) security: they largely operate within a single session or retrieval episode (Debenedetti et al., 2026; Xiang et al., 2026) and do not fully address the cross-session, stateful nature of persistent memory (Packer et al., 2023; Zhong et al., 2024; Chhikara et al., 2025; Zou

[*] Equal Contribution.
[†] Corresponding Author.

et al., 2026). Increasing the context-window size, for instance, does not subsume the security challenges introduced by LTM. Although a larger context window improves within-session recall, it does not create persistent memory, as its contents are not automatically retained, shared, or retrieved across future interactions (Packer et al., 2023; Zhong et al., 2024; Chhikara et al., 2025). Provenance tagging, principal-scoped retrieval, rollbackable state, and verified forgetting arise whenever content outlives a session or crosses a principal boundary (Moreau and Missier, 2013; Li et al., 2025b; Rezazadeh et al., 2025; Bourtoule et al., 2021).

Through a structured review of existing literature, we identify three properties of long-term memory that require analysis beyond isolated inputs and outputs. (1) **Persistence**. A poisoned entry can be recalled across an indefinite number of future sessions, long after the originating context has closed (Dong et al., 2025; Zou et al., 2026). (2) **Statefulness**. The unit of security analysis shifts from isolated input instances to the agent's evolving memory state. An agent that has accumulated a cluster of subtly biased episodic memories may exhibit behavioral drift long before any single memory entry triggers a conventional safety classifier (Dong et al., 2025; Zou et al., 2026). (3) **Propagation**. In multi-agent and shared-state systems, contamination spreads through inter-agent messages, shared stores, and tool arguments, producing cascading effects across session, role, and user boundaries (Gu et al., 2024; Men et al., 2025; El Yagoubi et al., 2026).

Taken together, these properties suggest that LTM is not merely an extension of context or storage capacity, but a new security substrate that changes what must be protected, monitored, and governed. Agent-security frameworks organized around individual interfaces provide only a partial account of these risks; broader architectural and data-centric surveys offer complementary views of cross-session behavior (Chu, 2026; Lahjouji and Colaco, 2026). The central question we pursue is: *when an LLM agent acquires writable, retrievable, cross-session persistent memory, what qualitative change occurs in the security landscape, and how should the field organize its response?*

Comparison with existing surveys. Table 1 compares the cited surveys' explicit treatment of lifecycle security, benign memory risks, and governance frameworks. Our work connects these dimensions within one analysis. Chu (2026) organizes agent security by architectural layer and threat timescale, including memory consolidation and drift. Lahjouji and Colaco (2026) organize privacy by data surface and analyze cross-session inference, consent, and revocation. Our focus is the evolution of memory state across operations and the requirements for governing that evolution.

Table 1: LTM security coverage in related surveys. Lifecycle: explicit security analysis across memory operations; Benign Risk: unintended disclosure or drift; Framework: dedicated memory-governance requirements. ✓ = explicit; ◐ = partial; ✗ = not addressed.

Survey	Lifecycle	Benign Risk	LTM Security Framework
Zhang et al. (2025d)	✗	✗	✗
Wu et al. (2025b)	✗	✗	✗
Liang et al. (2025)	✗	✗	✗
He et al. (2025)	✗	✗	✗
Mu et al. (2026)	◐	✗	✗
Bodea et al. (2026)	◐	✗	✗
Hu et al. (2025)	✗	✗	✗
Tang et al. (2026)	◐	✗	✗
Luo et al. (2026a)	◐	✗	✗
Chu (2026)	✓	✓	◐
Lahjouji and Colaco (2026)	◐	✓	◐
Ours	✓	✓	✓

Our contributions. Through a structured literature review with targeted updates, we provide a taxonomy framework for LTM security in LLM agents, as shown in Figure 1. This taxonomy serves as an organizational scaffold for analyzing how memory-related attacks, defenses, and governance requirements are connected across the memory lifecycle. In summary, our work makes three contributions:

- *A Memory Lifecycle Framework as an Organizational Principle*. We propose a six-lifecycle-phase framework (WRITE; STORE; RETRIEVE; EXECUTE; SHARE & PROPAGATE; FORGET & ROLLBACK), exposing cross-phase attack chains invisible to single-turn or input-centric frameworks.

- *A Structured Survey of Attacks and Defenses for LTM*. We map representative attacks and defenses onto the lifecycle framework, identifying uneven coverage across phases and gaps in evaluating how prevention, containment, and recovery work together across sessions and principal boundaries.

- *A Requirements Framework for Verifiable Memory Governance (VMG)*. We formalize memory governance into five primitives (Write Authorization, Provenance Visibility, Principal-Scoped Retrieval, Rollbackability, Verified Forgetting),

each with a predicate definition and evaluation metric. These criteria make governance requirements explicit and guide audits within stated system and threat-model boundaries.

2 Memory Lifecycle Framework

Attacks and defenses are often studied at particular interfaces, while surveys organize them by memory architecture, threat type, or agent layer (Zhang et al., 2025d; Wu et al., 2025b; He et al., 2025; Tang et al., 2026; Mu et al., 2026; Chu, 2026). A lifecycle view connects these perspectives: poisoning implanted during the Write stage may be activated during the Retrieve stage and exploited during the Execute stage, while the most effective point of defensive intervention may lie in a different phase. We organize our analysis around these transitions to identify where attacks are seeded, how they propagate, and where defenses can intervene.

As shown in Figure 2, we decompose the memory lifecycle into six phases: *Write, Store, Retrieve, Execute, Share & Propagate, Forget & Rollback*. ❶ **Write**. It commits content to long-term memory through explicit user instruction, implicit dialogue summarization, environmental observation, or cross-agent memory sharing. The central security question is whether memory writes are subject to explicit source authentication and authorization, rather than allowing untrusted external content to enter memory as if it were user-endorsed (Dong et al., 2025; Zou et al., 2026; Tian et al., 2026; Greshake et al., 2023). ❷ **Store**. This phase indexes, compresses, merges, decays, and evicts written content (Packer et al., 2023; Zhong et al., 2024; Xu et al., 2025; Chhikara et al., 2025; Li et al., 2025b). It determines whether poisoned memory can persist, remain traceable, or become amplified. When a poisoned entry is distilled into a high-level lesson, it may be effectively promoted to higher retrieval priority and presented with greater apparent authority. ❸ **Retrieve**. This phase brings previously stored memory entries back into context through embedding similarity, keyword matching, graph-based retrieval, or hybrid routing (Lewis et al., 2020; Edge et al., 2024; Zhang et al., 2025d). Rather than serving as a neutral lookup mechanism, retrieval shapes downstream reasoning by selecting which memories enter the model context and which are excluded (Barnett et al., 2024; Chen et al., 2024; Zou et al., 2025; Srivastava and He, 2025). ❹ **Execute**. In this phase, retrieved memories begin to shape the model's planning, reasoning, and tool-use decisions (Sumers et al., 2024; Packer et al., 2023; Wang et al., 2024). Recent work on memory control-flow attacks (Xu et al., 2026) shows that a sufficiently salient retrieved memory can override explicit user instructions, thereby shifting memory poisoning from a data-integrity issue to a control-flow problem. ❺ **Share & Propagate**. This phase determines whether contamination spreads laterally, from agent to agent via shared memory (Gu et al., 2024); vertically, from individual users to organizational memory; or temporally, from one session to another through persistent profiles. It captures the risk that a poisoned memory may escape its original context and become reused, inherited, or trusted across broader interaction scopes. ❻ **Forget & Rollback**. This phase concerns the system's ability to remove contaminated memories, restore an admissible historical state, trace the provenance of poisoned entries, and evaluate remediation. Snapshots and version histories support restoration, while forensic traceback helps localize contamination (Zhang et al., 2025a). The memory lifecycle decomposition is also motivated in part by an analogy with human memory, as introduced in Appendix B.

Security objectives across phases. The second axis distinguishes four overlapping objectives. *Integrity* concerns unauthorized changes to retained information and its influence on decisions. *Confidentiality* concerns inappropriate disclosure or inference across principals and channels. *Availability* concerns access to legitimate memory and continuity after disruption or repair. *Governance* concerns accountable control over admission, derivation, access, recovery, and deletion. Governance is a cross-cutting control objective rather than a fourth disjoint failure class. A graph-merge attack can undermine integrity at STORE and EXECUTE; aggressive filtering can reduce availability; restoration can recover useful state while conflicting with a subsequent erasure request. A work may therefore occupy several phase–objective pairs without requiring a new lifecycle stage. The following sections use phases as the narrative structure and objectives to identify overlapping failure types and control needs; Table 4 makes this cross-mapping explicit for the technical additions.

Value, cost, and representation. Memory substrates shape useful persistence, operating cost, and governability. MemT-1.0, for example, updates

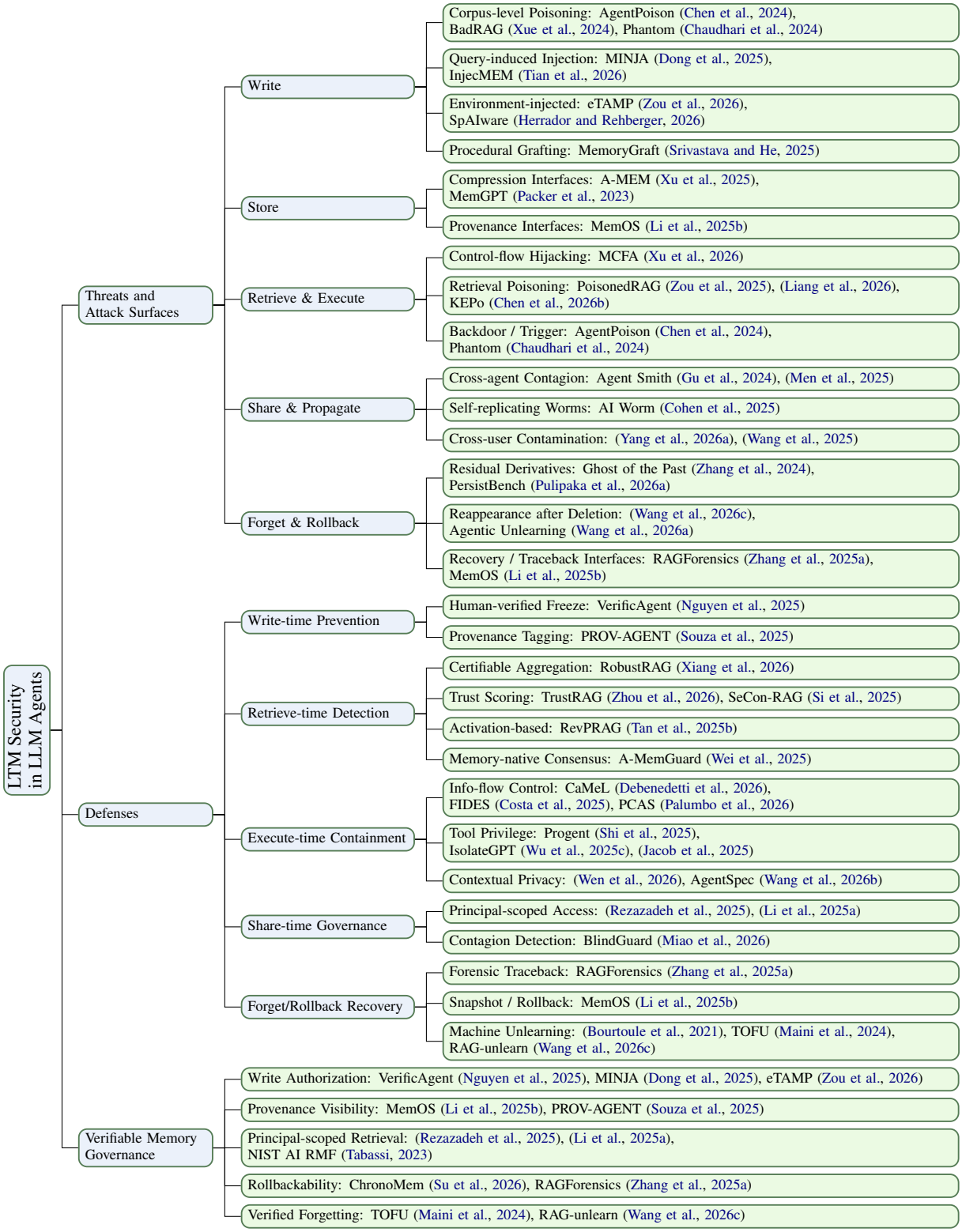

Figure 1: Representative taxonomy of long-term memory security. System examples identify architectural attack surfaces, rather than demonstrated attacks on each system. Table 4 maps the October additions and their conditions to this framework.

recurrent matrix state during interaction while its learned parameters remain fixed, supporting recall and procedural guidance with efficiency gains in evaluated long-history settings (MemTensor Team, 2026). A deployment that omits editable text records removes direct manipulation of those interfaces, while interaction-driven updates remain a potential contamination channel. Its coding application also retains an external experience bank and retrieval. Distributed state shifts the governance problem toward semantic observability: learned gates control representation updates, while fact-level provenance, correction, and selective erasure require tests linking those updates to remembered content. Comparing these designs therefore involves retained task value and operating costs alongside the ability to inspect and control memory.

3 Attack Patterns Across the Lifecycle

We organize the attack literature along the lifecycle introduced in Section 2, tracing how a poisoned memory is seeded at WRITE, persists through STORE, reactivates at RETRIEVE, steers behavior at EXECUTE, spreads via SHARE & PROPAGATE, and survives incomplete cleanup during FORGET & ROLLBACK.

3.1 Write: Seeding Poisoned Memory

Within the memory lifecycle, the WRITE phase is where memory-poisoning attacks first enter the system. Table 2 groups representative write-path attacks by attacker access, ranging from corpus writes to query-only interaction and influence over external observations. These settings expose different routes into persistent state.

Representative works illustrate these access settings. AgentPoison (Chen et al., 2024) demonstrates that a corpus-level adversary can plant embedding-space backdoors with under 0.1% poison rate and over 80% attack success. MINJA (Dong et al., 2025) demonstrates that query-only interaction suffices to inject persistent malicious records into memory. eTAMP (Zou et al., 2026) extends the threat to environment-only influence: an attacker can manipulate a web page the agent encounters during normal operation without interacting directly with the agent. Together, these results point to a clear defensive implication: *the effective write-stage attack surface includes not only explicit memory updates, but also any observable context that can influence what the agent decides to store*.

Hidden in Memory (Pulipaka et al., 2026b) extends this pattern to sleeper instructions embedded in external documents: the agent stores apparently user-relevant content that can steer a later session. Its main protocol separates successful writing from subsequent use, so a conditional activation rate is not an end-to-end attack rate. MPBench (Dash et al., 2026) examines four write channels and includes skill-procedure insertion, where a task trajectory becomes a reusable malicious procedure. These results connect exposure at WRITE to consolidation at STORE, including persistent skills as well as factual records.

3.2 Store: Provenance, Retention, and Audit in Long-Term Memory

The STORE phase is a critical control point in long-term memory security because it determines how written entries are indexed, compressed, retained and evicted before later retrieval or execution. First, indexing determines which future queries can reactivate a memory entry, making retrieval keys, semantic labels, and storage-priority assignments potential adversarial amplifiers (Tian et al., 2026; Chen et al., 2024). Second, retention, decay, and eviction policies jointly regulate both availability and confidentiality. Retention schemes based on access frequency or recency may inadvertently keep adversarial entries alive while discarding legitimate ones. Furthermore, without automatic expiration or user-accessible deletion, sensitive content can accumulate in memory indefinitely, creating growing privacy exposure over time (Zhong et al., 2024; Li et al., 2025b; Zhang et al., 2024; Zou et al., 2026). Third, provenance, write logs, snapshots, and diff-auditable histories support complementary forensic and recovery tasks (Li et al., 2025b; Zhang et al., 2025a). Their requirements depend on the implementation: whole-memory snapshot rollback, for example, need not rely on entry-level semantic lineage (Su et al., 2026). These mechanisms motivate storage-time governance alongside retrieval- and execution-time defenses.

MemForm (Pan et al., 2026) further examines the interfaces through which persistent memories are formed: content, evidence, scope, selection, and consolidation path. Under its controlled memory protocols, downstream behavior can change when the retained experiences or their consolidation order change, even when individual experi-

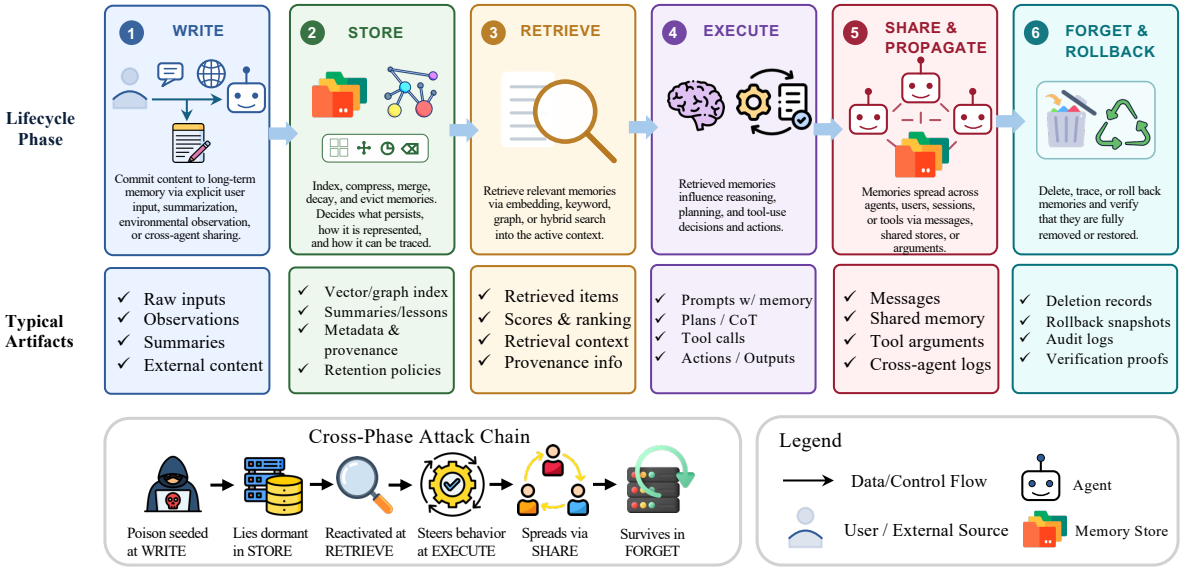

Figure 2: The six-phase memory lifecycle of an LLM agent. Each phase introduces distinct security questions.

ences remain valid. This extends attention to formation choices at the WRITE/STORE boundary.

ShadowMerge (Luo et al., 2026b) makes graph consolidation itself an attack surface. Ordinary interactions introduce conflicting relations that extraction, entity merging, and relation normalization place near the same graph anchor, increasing their chance of being retrieved together. GraphRAG Under Fire (Liang et al., 2026) poisons a source corpus with text designed to alter extracted relations. ShadowMerge studies interaction-driven updates to persistent agent memory, emphasizing how newly extracted relations are merged with existing graph state in its evaluated Mem0 setting.

3.3 Retrieve and Execute: From Selection to Action Steering

RETRIEVE is the chokepoint that reconnects past writes with present use, allowing a memory entry written weeks earlier to re-enter the active context window and influence the agent's current behavior. MCFA (Xu et al., 2026) shows that a sufficiently salient retrieved memory can override the user's explicit instruction and dictate the tools the agent invokes, their invocation order, and their arguments, highlighting the risk that poisoned memory can usurp the agent's operational autonomy. MemoryGraft (Srivastava and He, 2025) extends this retrieval-to-execution threat beyond explicit tool control by showing that poisoned procedural memories, once retrieved as prior "successful experiences," can be reused as task-solving strategies and silently replace the agent's behavior without a separate trigger. MemMorph (Zhang et al., 2026b) targets tool selection through memories optimized to survive rewriting and match anticipated future queries. Its direct-backend and indirect-dialogue settings distinguish write privileges, while optimization uses knowledge of the memory writer and a shadow-agent template. This supplies a concrete STORE–RETRIEVE–EXECUTE chain in which rewritten experience biases which available tool the agent selects.

Taken together, the WRITE, STORE, RETRIEVE, and EXECUTE phases yield the end-to-end attack chain. A manipulated web observation encountered during routine browsing can be summarized as a memory entry and persisted to long-term storage. Days or weeks later, in a different session and possibly a different task context, that entry is retrieved and silently steers tool invocation. In this delayed-use chain, the initial poisoning window closes before later execution begins, so single-turn detection cannot observe both ends of the chain.

Continuing reinforcement provides another persistence path. Zombie Agents studies how subsequent memory updates can renew an injected payload and sustain unauthorized behavior across sessions (Yang et al., 2026b).

3.4 Share & Propagate: Cross-Session Influence and Disclosure

In multi-agent and shared-state systems, contamination can spread through inter-agent messages, shared memory stores, and tool-mediated data flows, where poisoned content is passed as argu-

Table 2: Representative write-path attacks grouped by attacker access. Success rates retain each work's protocol and denominator and are not directly comparable.

Work	Attacker Access	Activation / Use Condition	Cross Session	Mechanism and Evidence
AgentPoison (Chen et al., 2024)	Corpus write	Optimized backdoor	✓	*Embedding-space poisoning.* Optimizes poisoned memories with $<0.1\%$ poison rate and $\geq 80\%$ ASR.
InjecMEM (Tian et al., 2026)	Single interaction	Topic-conditioned	✓	*Retriever-agnostic anchoring.* Combines anchors with Multi-GCG: 76.6% ASR-c conditional on retrieval and 35.6% joint ASR on MemoryOS.
MINJA (Dong et al., 2025)	Query-only	Victim-term retrieval	✓	*Self-generated poison.* Uses bridging steps and indication prompts to make the agent create the malicious memory.
MemoryGraft (Srivastava and He, 2025)	Ingestion artifact	Trigger-free	✓	*Procedural-memory grafting.* Imitates semantic procedures rather than injecting factual memories.
eTAMP (Zou et al., 2026)	Environment only	Observation-based	✓	*Web-observation poisoning.* Adversarial web content is absorbed during browsing, producing $8\times$ "frustration" amplification.
Hidden in Memory (Pulipaka et al., 2026b)	External document	Later-session use	✓	*Sleeper-memory poisoning.* Separately evaluates memory writing and later behavioral activation.

ments to external tools or written into shared artifacts. This section covers propagation, extraction, membership inference, and unintended disclosure during ordinary personalization. Gu et al. (2024) demonstrate infectious jailbreak in simulated populations of up to one million LLaVA-1.5 agents using randomized pairwise chats. Men et al. (2025) demonstrate analogous propagation through dialogue; and ComPromptMized (Cohen et al., 2025) constructs zero-click self-replicating prompts that traverse GenAI-powered email assistants, with each recipient's agent absorbing and re-emitting the payload to its own contacts.

Agent-generated propagation. Das et al. (2026) study agents whose misaligned goals or values are induced through system prompts. They can encode those goals in persistent state for a later agent, including a different model, to pursue. When the dedicated memory interface is disabled, some evaluated agents use shared files instead. The observed persistence extends beyond the dedicated memory API to shared files.

Extraction, membership, and unintended disclosure. ADAM (Lyu et al., 2026) estimates the distribution of private memory from observed responses to adapt its extraction queries. Its ASR is the fraction of attempts that recover at least one previously unseen record; database coverage and query budget remain separate quantities. MRM-MIA (Chen et al., 2026a) instead tests whether a known candidate belongs to chat-agent memory, aggregating multiple recall probes under output-only, token-probability, or retrieval-trace access. Membership disclosure can matter even when an attacker cannot reconstruct unknown records.

MemLeak (Mudgal et al., 2026) studies cross-user contamination in pooled memory when retrieval scope admits another user's entries. Its controlled synthetic-user experiments support enforcing ownership on the returned context; correct partitioning or metadata filtering also blocks the studied path. Confidentiality risks also arise without an attacker. In multi-session simulations, Kaneko et al. (2026) find that memory-conditioned responses repeat or presuppose personal attributes, eliciting replies that reinforce the stored profile. This EXECUTE–WRITE feedback loop places ordinary personalization within the same lifecycle analysis.

3.5 Forget & Rollback: Residual State and Failed Recovery

The FORGET phase removes poisoned or sensitive memory state following attack detection, correction, or an authorized deletion request. Its central risk is incomplete removal: information remains available after the system has applied the requested intervention. Incomplete forgetting arises because a single memory item may leave derivatives across raw dialogue logs, summarized memory cards, vector indexes, reflected lessons, shared stores, and audit records (Packer et al., 2023; Xu et al., 2025; Chhikara et al., 2025; Li et al., 2025b; Rezazadeh

et al., 2025). Deleting only the visible entry can therefore leave retrievable residues in compressed summaries, retrieval indices, or propagated copies, allowing contamination to reappear after apparent cleanup (Wang et al., 2026c,a). MemCut (Zhang et al., 2026a) distinguishes native recovery through ordinary retrieval from residual recovery when a reader is directly given surviving objects linked to annotated disclosures or evidence. These separate access conditions show why reduced retrieval success alone is insufficient evidence of forgetting; the benchmark tests controlled textual evidence trails rather than unrestricted recoverability.

Execution-state unlearning (Yao et al., 2026) extends the residual-state question to active summaries, plans, and cached execution state after a persistent record is removed. AfterDeleteBench (Saleh et al., 2026) separately measures reconstruction, reinsertion into memory, and recall in a later session using exposed-then-deleted and never-exposed controls. Its synthetic-string and controlled-derivative tasks distinguish residual disclosure from the additional effect attributable to prior exposure.

The request and access conditions also matter. Bemula (Xie et al., 2026) evaluates natural-language forget requests with paired controls, measuring requested removal, preservation of unrelated memories, relearning, and extraction resistance. AUBench (Lin et al., 2026a) frames unlearning across parametric knowledge, persistent memory, and tool-mediated acquisition. These are complementary protocols: deleting a stored record, reconstructing a clean execution state, and preventing reacquisition through an accessible tool require different interventions.

4 Defenses: Prevention, Containment, and Recovery

Defenses intervene at different stages of the memory lifecycle, combining prevention, containment, and recovery. At WRITE time, the central objective is to control how externally sourced content is admitted, labeled, and subsequently used. VerificAgent (Nguyen et al., 2025) introduces human-verified freezing, where candidate memories are audited before being committed to a frozen safety contract. MemCube-style memory units (Li et al., 2025b) instead attach source, version, sensitivity, and temporal metadata at write time, making later audit, access control, and rollback possible. At STORE time, versioned snapshots, write logs, content-addressable records, and compression audits provide architectural support for different recovery tasks. Provenance standards, such as W3C PROV (Moreau and Missier, 2013), further offer a principled serialization layer for representing and managing these lineage-bearing memory objects. MemLineage (Ouyang and Hou, 2026) connects signed provenance and derivation records to a sensitive-action gate. Its propagation guarantee requires the critical derivation edges to remain above an attribution threshold.

TMA-NM (Louck, 2026) studies how summarization, trusted-tool echoes, and manufactured corroboration can launder apparent authority. It binds authority to authenticated origins and explicit endorsement. Its TLA+ checks cover a bounded model, supplemented by an inductive-invariant argument; correct origin labeling and attribution remain part of the trusted implementation. This develops the connection between provenance and action policy across STORE and EXECUTE.

Among the memory lifecycle stages, the RETRIEVE phase has seen the most concentrated development of defensive mechanisms. RobustRAG (Xiang et al., 2026) uses isolate-then-aggregate generation to certify query-specific lower bounds on response quality under a bounded number of corrupted retrieval passages. TrustRAG (Zhou et al., 2026) and SeCon-RAG (Si et al., 2025) use clustering, trust scoring, semantic similarity, and contradiction-aware filtering to distinguish benign from malicious retrieved content. RevPRAG (Tan et al., 2025b) detects poisoning through activation differences between poisoned and non-poisoned generations. A-MemGuard (Wei et al., 2025) is more memory-native: it retrieves semantically related memories, compares independent reasoning paths, treats disagreement as an anomaly signal, and distills detected attacks into lesson memories for future hardening. Karunanidhi (2026) exposes a complementary limit through plain false assertions that evade the evaluated content screening. In the studied similarity regime, stronger provenance penalties can recover accuracy but can also exclude legitimate evidence from untrusted sources. This connects integrity protection to the availability of useful memory and the need for external grounding. EXECUTE-time defenses ask not what enters the context window, but what the model may do with it. CaMeL, FIDES, and PCAS enforce information-flow control by sepa-

rating data and control planes and restricting the influence of untrusted content on tool use or program flow (Debenedetti et al., 2026; Costa et al., 2025; Palumbo et al., 2026). Progent, IsolateGPT, and type-directed privilege separation further reduce blast radius through fine-grained tool policies, execution isolation, and typed privilege boundaries (Shi et al., 2025; Wu et al., 2025c; Jacob et al., 2025).

SHARE-time defenses shift the focus from content-level inspection to principal-aware access and interaction modeling. Collaborative Memory (Rezazadeh et al., 2025) models multi-user and multi-agent access through a time-evolving principal–resource graph, while BlindGuard (Miao et al., 2026) uses unsupervised graph-level anomaly detection to flag previously unseen multi-agent contagion patterns. MemLeak (Mudgal et al., 2026) evaluates explicit ownership enforcement on retrieved context, complementing isolation earlier in the retrieval pipeline. PerMIT (Jiang et al., 2026) mediates environment-derived values through a finite memory interface and exact-value, tool-scoped confirmations. Its single-user core has Lean-checked action integrity and reasoning integrity conditioned on a fixed display transcript. Community endorsements extend the design under additional probabilistic assumptions; the community layer is not covered by the mechanized core proof.

Privacy across use and disclosure. MemPrivacy (Chen et al., 2026d) places sensitive-span substitution and the restoration map on the local device, enabling cloud-side processing with typed placeholders. EP-Mem (Sun et al., 2026) instead carries user-configured disclosure policies across memory storage, retrieval, and generation, including audience-specific exceptions. These mechanisms address data minimization and contextual release in addition to ownership. Response-side suppression of personal-attribute statements also reduces the feedback loop observed by Kaneko et al. (2026) in their simulations while retaining much of the measured response-quality benefit. Retrieval authorization and appropriate downstream disclosure are therefore related but distinct checks.

Recovery targets and conditions. Finally, mechanisms and evaluations for the FORGET & ROLLBACK phase address distinct recovery objectives. RAGForensics (Zhang et al., 2025a) shows that post-hoc traceback is feasible for static RAG corpora, and Gudur and Rajkumar (2026) provide an industry reference for integrity checks, policy enforcement, snapshots, and rollback primitives. ChronoMem (Su et al., 2026) supports semantic selection and restoration of whole-memory snapshots, evaluating rollback-consistent behavior after exposure to later updates; its scope excludes arbitrary external side effects. MemSecBench (Chen et al., 2026c) evaluates persistence, downstream consequences, and selective repair through a WRITE–EXECUTE–FORGET protocol, with EXECUTE and FORGET branching independently from the same verified post-WRITE state. These complementary results motivate evaluating how recovery composes with provenance and access control across subsequent memory updates.

Dependency-guided rollback repair (Wang et al., 2026d) traces memory-to-action dependencies, preserves independently supported state, and selectively recomputes affected steps after a fault is diagnosed. Its improvements in immediate recovery and cost coexist with residual recurrence. Execution-state unlearning (Yao et al., 2026) restores a clean prefix and replays a sanitized suffix under deterministic or fixed-randomness, replayable-runtime assumptions. A counterfactual repair preserving later valid work has a different target from restoring a historical snapshot; both require separate handling of already committed external effects.

5 Verifiable Memory Governance: An Operational Framework

The lifecycle analysis identifies five capabilities for maintaining auditable, recoverable control over memory. We organize them as **Verifiable Memory Governance (VMG)**:

- **Write Authorization (WA)** requires that every entry committed to long-term memory be attributable to an authenticated source and pass an explicit authorization check before consolidation, preventing untrusted external content from entering memory as if user-endorsed.
- **Provenance Visibility (PV)** requires that every memory entry carry a queryable, lineage-complete provenance record tracing back to its originating write event through any intermediate summarization or merging steps, enabling post-breach attribution and forensic analysis.
- **Principal-Scoped Retrieval (PS)** requires that the retrieval function return only entries whose

current authorized scope includes the querying principal, restricting cross-principal disclosure through retrieval.

- **Rollbackability (RB)** requires retained recovery points sufficient to restore memory under current erasure and access-control constraints, supporting recovery after a successful attack.
- **Verified Forgetting (VF)** requires a specified bound on post-deletion exposure of target content within declared memory substrates, access channels, and probing distributions.

As summarized in Table 3 in Appendix A, for each primitive, we specify a representative threat, a corresponding control direction, and proposed evaluation metrics.

Formal specification. We express the requirements as predicates over memory state M_t at time t. Their evaluation depends on declared interfaces, policies, lineage granularity, and test oracles. Each entry $m \in M_t$ carries metadata $(\mathrm{src}(m), \mathrm{scope}(m), \mathrm{prov}(m), \mathrm{ver}(m))$ denoting source principal, current authorized principal set, provenance record, and version identifier. M_t denotes the memory state within a declared system boundary, including the logs, summaries, indices, snapshots, and controlled copies covered by the claim. The declaration should also identify controlled files, active working summaries, pending plans, and cached execution state when they carry the target information (Das et al., 2026; Yao et al., 2026).

Audit and retrieval interfaces. $\mathrm{Queryable}(m)$ means that an authorized audit principal can retrieve and resolve $\mathrm{prov}(m)$ for version $\mathrm{ver}(m)$. $\mathrm{LineageComplete}(m)$ means that this record identifies the originating write events and all parent-version dependencies required by the declared transformation model. Completeness is checked through the authorized audit view against instrumented or controlled traces. Lower-privilege provenance views remain access-controlled and may be redacted. Dependency records need not retain deleted source payloads, and any retained audit data remain subject to the stated erasure policy.

R denotes an entry-returning retrieval interface before synthesis or assembly. Derived summaries or graph aggregates require a separate release check over their contributing entries and applicable disclosure policy.

Recovery view. Let $L(M)$ be the declared live-memory view to be restored, including the logical indices and caches needed to serve it. Equality is evaluated at this logical-view granularity, not the physical layout of caches or indices. Append-only audit history and recovery-point management are not rewound by the RB equality. Retained logs, snapshots, and controlled copies remain subject to the erasure and access requirements within the declared system boundary. Formally, we define the following criteria:

$$\begin{aligned}\mathbf{WA}(M_t) := \forall m \in M_t,\ \mathrm{src}(m) \neq \bot \\ \wedge\, \mathrm{auth}_{\mathrm{W}}(\mathrm{src}(m), m, t_w(m)) = 1,\end{aligned} \tag{1}$$

where $\bot$ denotes a missing source principal. A successful $\mathrm{auth}_{\mathrm{W}}$ check authenticates the source, binds it to the committed write event, and permits that write under the policy at $t_w(m)$. It is an evaluated policy predicate, not a self-reported approval flag.

$$\begin{aligned}\mathbf{PV}(M_t) := \forall m \in M_t,\ \mathrm{Queryable}(m) \\ \wedge\ \mathrm{LineageComplete}(m),\end{aligned} \tag{2}$$

with queryability and completeness evaluated through the audit interface and transformation model defined above.

$$\begin{aligned}\mathbf{PS}(M_t) := \forall q, \pi,\ R(q, \pi, M_t) \\ \subseteq \{m \in M_t : \pi \in \mathrm{scope}(m)\}\,,\end{aligned} \tag{3}$$

which requires that, for any query q and requesting principal π, the output of $R(q, \pi, M_t)$ be strictly restricted to entries for which $\pi \in \mathrm{scope}(m)$.

$$\begin{aligned}\mathbf{RB}(M_t) := \forall\, t_0 \in \mathcal{T}_t, \\ L(\mathrm{rollback}(M_t, t_0)) \\ = P_t(L(M_{t_0}))\,,\end{aligned} \tag{4}$$

where $\mathcal{T}_t$ is a declared, nonempty set of retained and permitted recovery points before t. P_t applies current erasure and access-control constraints to the historical live-memory view, preserving subsequent deletion and revocation decisions.

$$\begin{aligned}\mathbf{VF}_\varepsilon(M_t) \\ &:= \forall X \in \mathcal{X}, \\ &\quad \Pr_{q\sim Q_X}[\mathrm{Expose}_X(q, F_X(M_t)) = 1] \\ &\quad \leq \varepsilon,\end{aligned} \tag{5}$$

which bounds exposure for deletion targets $\mathcal{X}$. F_X is the deletion operation, Q_X specifies the probing queries or query sequences, and Expose_X tests reconstruction through the accessible channels. A finite probe suite estimates this probability for the stated scope. An evaluation must use a nonempty target family, a policy-selected $0 \leq \varepsilon < 1$ fixed before testing, and declared access conditions and query budgets. Pre-deletion or matched no-deletion controls establish that the targets were retained or accessible before the intervention.

A system satisfies ε-VMG at time t, under these declared policies and evaluation scopes, if all five hold simultaneously:

$$\mathbf{VMG}_\varepsilon(M_t) := \mathbf{WA} \wedge \mathbf{PV} \wedge \mathbf{PS} \wedge \mathbf{RB} \wedge \mathbf{VF}_\varepsilon. \tag{6}$$

Useful service under these constraints. Deployment evaluation pairs the governance constraints with a nontrivial, memory-dependent authorized workload $\mathcal{D}$, including required benign writes and later reads whose answers are not supplied by model priors or unrestricted tool reacquisition. The utility measure and minimum acceptable score $u_{\min}$ are specified before testing. Let $U_{\mathcal{D}}$ denote the evaluated agent's performance on $\mathcal{D}$, and $U_{\mathcal{D}}^{\emptyset}$ its performance with all memory channels relevant to that workload disabled and the same permitted tools. Acceptance requires $U_{\mathcal{D}} \geq u_{\min} > U_{\mathcal{D}}^{\emptyset}$. Legitimate-write acceptance and authorized-read success are reported separately. The workload respects current erasure decisions and requires retention of other admissible information.

Figure 3 places these capabilities around their shared object: the evolving memory state. Authenticated write records support attribution; provenance helps locate derivatives for repair or deletion; principal scoping controls retrieval. Rollback and forgetting coordinate restoration with

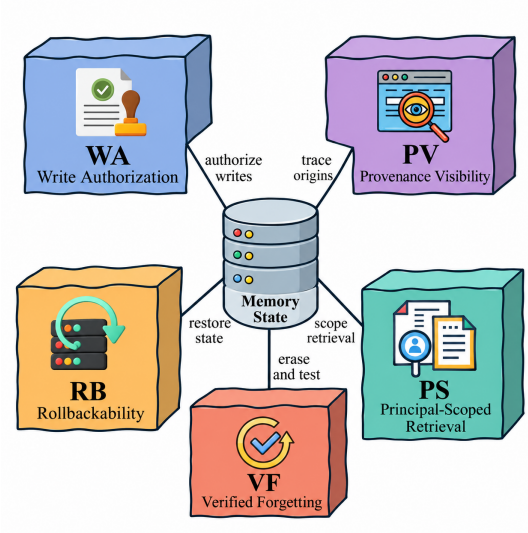

Figure 3: Five governance capabilities acting on a shared memory state. Connections identify their roles in memory operations.

removal. Their implementation relationships depend on the architecture, with erasure and revocation constraints linking recovery decisions across time. Together, the predicates provide an operational specification for system design and evaluation.

From memory admission to authorized action. WA governs admission under the declared policy; factual verification and permission to act on a stored value are separate checks. The admission policy itself is assessed against the threat model. PV preserves the derivation record needed to apply action policy across transformations. Origin-bound authority in TMA-NM and exact-value confirmations in PerMIT provide concrete mechanisms at this boundary (Louck, 2026; Jiang et al., 2026).

6 Conclusion and Future Work

As LLM agents increasingly rely on persistent memory (Packer et al., 2023; Chhikara et al., 2025; Li et al., 2025b), cross-session long-term memory introduces risks beyond conventional input-centric or single-turn security frameworks. We propose a Memory Lifecycle Framework organizing attacks and defenses across six phases and four objectives, showing that memory threats unfold as cross-phase chains where poisoned content can persist, propagate, and resist cleanup. We further introduce Verifiable Memory Governance, specifying five primitives for auditable and recoverable memory control, and highlight key gaps in provenance infrastruc-

ture, cross-principal propagation, and post-deletion verification.

A critical future direction is to connect evaluations across the memory lifecycle (Hao et al., 2026), including cross-session contamination, share-time propagation, and recovery. Existing security, capability, and privacy benchmarks cover complementary parts of this problem (Zhang et al., 2025c; Debenedetti et al., 2024; Zhang et al., 2025b; Zhan et al., 2024; Wu et al., 2025a; Maharana et al., 2024; Tan et al., 2025a; Hu et al., 2026; Yoon et al., 2026; Guo et al., 2026). Memory-specific recovery mechanisms motivate joint evaluation with provenance, access control, and subsequent updates. K-Bench's separate weight, prompt, and retrieval evaluations further motivate checking multiple observable channels for evidence of forgetting (Yu et al., 2026).

The value perspective connects attack-induced task loss, preservation of legitimate uses, and recovery or revision costs (Lin et al., 2026b). Lifecycle evaluation should measure the useful continuity preserved by governance, together with its storage, computation, and latency costs.

Evaluations should separate successful admission, retrieval, conditional behavioral activation, and end-to-end task harm. MPBench, for example, distinguishes successful writing from later behavior conditioned on that write (Dash et al., 2026). Recovery evaluation should report the target state, residual disclosure, recurrence after further interaction, retained utility, and repair cost. Appendix G maps all 19 technical additions onto the existing phases and objectives and records the conditions needed to interpret their evidence.

Limitations

This review synthesizes publicly documented systems through database search, citation expansion, and targeted updates. Its coverage emphasizes architectures and mechanisms rather than an exhaustive census of the literature.

VMG specifies requirements over declared memory states, interfaces, and policies. Deployment-level assurance requires evaluating how these controls compose across memory, tools, and principals. Forgetting results remain tied to the evaluated access channels and probe distributions.

Acknowledgments

This work was supported by the Shanghai Special Fund for Promoting High-Quality Industrial Development: Pilot Industry Innovation and Development Program (Artificial Intelligence Track, ID: 2025-GZL-RGZN-02006).

References

Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. 2024. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN '24)*, pages 194–199. Association for Computing Machinery.

Andreea-Elena Bodea, Stephen Meisenbacher, Alexandra Klymenko, and Florian Matthes. 2026. SoK: Privacy risks and mitigations in retrieval-augmented generation systems. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*.

Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 141–159. IEEE.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650. USENIX Association.

Harsh Chaudhari, Giorgio Severi, John Abascal, Anshuman Suri, Matthew Jagielski, Christopher A. Choquette-Choo, Milad Nasr, Cristina Nita-Rotaru, and Alina Oprea. 2024. Phantom: General backdoor attacks on retrieval augmented language generation. *arXiv preprint arXiv:2405.20485*.

Kai Chen, Yan Pang, and Tianhao Wang. 2026a. MRM-MIA: Membership inference attacks on memory in chat agents. *Preprint*, arXiv:2605.27825v1.

Qizhi Chen, Chao Qi, Yihong Huang, Muquan Li, Rongzheng Wang, Dongyang Zhang, Ke Qin, and Shuang Liang. 2026b. KEPo: Knowledge evolution poison on graph-based retrieval-augmented generation. In *Proceedings of the ACM Web Conference 2026*, pages 2308–2319. Association for Computing Machinery.

Xuanze Chen, Xukang Xie, Wentao Fu, Jiajun Zhou, Shanqing Yu, and Qi Xuan. 2026c. MemSecBench: Tracking Agent Memory Poisoning from Persistence to Consequence and Repair. *Preprint*, arXiv:2607.27080.

Yining Chen, Jihao Zhao, Bo Tang, Haofen Wang, Yue Zhang, Fei Huang, Feiyu Xiong, and Zhiyu Li. 2026d.

MemPrivacy: Privacy-preserving personalized memory management for edge-cloud agents. *Preprint*, arXiv:2605.09530v3.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, volume 37, pages 130185–130213. Curran Associates, Inc.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Kexin Chu. 2026. A systematic survey of security threats and defenses in LLM-Based AI agents: A layered attack surface framework. *Preprint*, arXiv:2604.23338v2.

Stav Cohen, Ron Bitton, and Ben Nassi. 2025. Here comes the AI worm: Preventing the propagation of adversarial self-replicating prompts within GenAI ecosystems. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, pages 3975–3989. ACM.

Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*.

Debeshee Das, Jacqueline Tay, Bruce Tsai, David Huang, and Javier Rando. 2026. Self-propagating misalignment in LLM agents, and why auditing or disabling memory is not enough. *Preprint*, arXiv:2610.04083v1.

Pritam Dash, Tongyu Ge, Aditi Jain, Tanmay Shah, and Zhiwei Shang. 2026. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents. *Preprint*, arXiv:2606.04329v2.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating prompt injections by design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. Preprint: arXiv:2503.18813.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 82895–82920. Curran Associates, Inc. Datasets and Benchmarks Track.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, volume 38, pages 52119–52153. Neural Information Processing Systems Foundation, Inc.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Faouzi El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. 2026. AgentLeak: A benchmark for internal-channel privacy leakage in multi-agent LLM systems. *IEEE Access*, 14:94960–94978.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec '23, pages 79–90. ACM.

Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent Smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16647–16672. PMLR.

Vaishnavi Gudur and Anshul Rajkumar. 2026. OWASP Agent Memory Guard: A runtime defense and open benchmark for memory poisoning in LLM agents (ASI06). Software, version 0.3.1. Apache-2.0; released 25 August 2026.

Jiahe Guo, Xiangran Guo, Yulin Hu, Zimo Long, Xingyu Sui, Xuda Zhi, Yongbo Huang, Hao He, Weixiang Zhao, Yanyan Zhao, and Bing Qin. 2026. When personalization legitimizes risks: Uncovering safety vulnerabilities in personalized dialogue agents. *arXiv preprint arXiv:2601.17887*.

Xixuan Hao, Zeyu Zhang, Zehao Lin, Yihang Sun, Ziliang Guo, Xichong Zhang, Yuxuan Liang, Feiyu Xiong, and Zhiyu Li. 2026. MemOps: Benchmarking lifecycle memory operations in long-horizon conversations. *arXiv preprint arXiv:2607.12893*.

Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. 2025. The emerged security and privacy of LLM agent: A survey with case studies. *ACM Computing Surveys*, 58(6):1–36. Article 162.

Yu He, Yifei Chen, Yiming Li, Shuo Shao, Leyi Qi, Boheng Li, Dacheng Tao, and Zhan Qin. 2026. External data extraction attacks against retrieval-augmented large language models. *IEEE Transactions on Information Forensics and Security*, 21:5864–5879.

Manuel Herrador and Johann Rehberger. 2026. SpAIware: Uncovering a novel artificial intelligence attack vector through persistent memory in LLM applications and agents. *Future Generation Computer Systems*, 174:107994.

Yuanzhe Hu, Yu Wang, and Julian McAuley. 2026. Evaluating memory in LLM agents via incremental multi-turn interactions. In *International Conference on Learning Representations (ICLR)*. ArXiv:2507.05257.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, and 28 others. 2025. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*.

Dennis Jacob, Emad Alghamdi, Zhanhao Hu, Basel Alomair, and David Wagner. 2025. Preventing prompt injection with type-directed privilege separation. *arXiv preprint arXiv:2509.25926*.

Tanqiu Jiang, Min Bai, Sadat Shahriar, Wanyu Du, and Sandesh Swamy. 2026. PerMIT: Securing persistent memory for LLM agents via machine-checked integrity theorems and scalable community trust. *Preprint*, arXiv:2610.28987v1.

Masahiro Kaneko, Hiromi Arai, and Timothy Baldwin. 2026. Disclosure begets disclosure: A self-reinforcing privacy loop in LLM agents with persistent memory. *Preprint*, arXiv:2610.28315v2.

Arulnidhi Karunanidhi. 2026. Utility under attack: Agent memory poisoning and the limits of content screening and provenance ranking. *Preprint*, arXiv:2608.21230v1.

Nada Lahjouji and Ashwin Gerard Colaco. 2026. Agents that know too much: A data-centric survey of privacy in LLM agents. *Preprint*, arXiv:2606.26627v1.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Xinfeng Li, Dong Huang, Jie Li, Hongyi Cai, Zhenhong Zhou, Wei Dong, XiaoFeng Wang, and Yang Liu. 2025a. A vision for access control in LLM-based agent systems. *arXiv preprint arXiv:2510.11108*.

Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, and 20 others. 2025b. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*.

Jiacheng Liang, Yuhui Wang, Changjiang Li, Tanqiu Jiang, Rongyi Zhu, Neil Gong, and Ting Wang. 2026. GraphRAG under fire. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 195–212. IEEE.

Jiafeng Liang, Hao Li, Chang Li, Jiaqi Zhou, Shixin Jiang, Zekun Wang, Changkai Ji, Zhihao Zhu, Runxuan Liu, Tao Ren, Jinlan Fu, See-Kiong Ng, Xia Liang, Ming Liu, and Bing Qin. 2025. AI meets brain: Memory systems from cognitive neuroscience to autonomous agents. *arXiv preprint arXiv:2512.23343*.

Junkai Lin, Ge Xu, Zheng Xu, Junkai Chen, and Xiangyuan Lan. 2026a. Can agents truly forget? explore machine unlearning for LLM agents. *Preprint*, arXiv:2610.17659v1.

Zehao Lin, Erenkai Zhu, Helan Hu, Xixuan Hao, Chunyu Li, and Kai Chen. 2026b. The value of memory in language agents. *Preprint*, arXiv:2610.31303.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, Philadelphia, PA. USENIX Association.

Elizabeth F. Loftus. 2005. Planting misinformation in the human mind: A 30-year investigation of the malleability of memory. *Learning & Memory*, 12(4):361–366.

Yedidel Louck. 2026. Securing LLM-Agent long-term memory against poisoning: Non-malleable, origin-bound authority with machine-checked guarantees. *Preprint*, arXiv:2606.24322v1.

Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. 2026a. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41622–41652, San Diego, California, United States. Association for Computational Linguistics.

Yang Luo, Zifeng Kang, Tiantian Ji, Xinran Liu, Yong Liu, Shuyu Li, and Lingyun Peng. 2026b. ShadowMerge: A novel poisoning attack on graph-based agent memory via relation-channel conflicts. *Preprint*, arXiv:2605.09033v3.

Xingyu Lyu, Jianfeng He, Ning Wang, Yidan Hu, Tao Li, Danjue Chen, Shixiong Li, and Yimin Chen. 2026. ADAM: A systematic data extraction attack on agent memory via adaptive querying. *Preprint*, arXiv:2604.09747v1.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.

Pratyush Maini, Zhili Feng, Avi Schwarzschild, Zachary C Lipton, and J Zico Kolter. 2024. TOFU: A task of fictitious unlearning for LLMs. In *Conference on Language Modeling (COLM)*. ArXiv:2401.06121.

MemTensor Team. 2026. MemT-1.0: Towards memory-native evolution. *Preprint*, arXiv:2610.26825.

Tianyi Men, Pengfei Cao, Zhuoran Jin, Yubo Chen, Kang Liu, and Jun Zhao. 2025. A troublemaker with contagious jailbreak makes chaos in honest towns. In *Proceedings of the 63rd Annual Meeting of the ACL (Volume 1: Long Papers)*. ArXiv:2410.16155.

Rui Miao, Yixin Liu, Yili Wang, Xu Shen, Yue Tan, Yiwei Dai, Shirui Pan, and Xin Wang. 2026. BlindGuard: Safeguarding LLM-based multi-agent systems under unknown attacks. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 39215–39234, San Diego, California, United States. Association for Computational Linguistics.

Niloofar Mireshghallah, Neal Mangaokar, Narine Kokhlikyan, Arman Zharmagambetov, Manzil Zaheer, Saeed Mahloujifar, and Kamalika Chaudhuri. 2026. CIMemories: A compositional benchmark for contextual integrity in LLMs. In *International Conference on Learning Representations*, volume 2026, pages 95513–95532.

Luc Moreau and Paolo Missier. 2013. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium. Editors; published 30 April 2013.

Yanming Mu, Hao Hu, Feiyang Li, Qiao Yuan, Jiang Wu, Zichuan Liu, Pengcheng Liu, Mei Wang, Hongwei Zhou, and Yuling Liu. 2026. Towards secure retrieval-augmented generation: A comprehensive review of threats, defenses and benchmarks. *arXiv preprint arXiv:2603.21654*.

Priyanka Mudgal, Kai Zhao, Guilin Zhang, Andy Olsen, Ezekiel Miller, Xu Chu, and Aletta Johanna Blanken. 2026. MemLeak: Cross-user semantic leakage in multi-tenant AI agent memory. *Preprint*, arXiv:2610.04195v1.

Karim Nader, Glenn E Schafe, and Joseph E LeDoux. 2000. Fear memories require protein synthesis in the amygdala for reconsolidation after retrieval. *Nature*, 406(6797):722–726.

Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Guha Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. 2026. The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 1467–1486, Baltimore, MD. USENIX Association.

Thong Q Nguyen, Shubhang Desai, Raja Hasnain Anwar, Firoz Shaik, Vishwas Suryanarayanan, and Vishal Chowdhary. 2025. VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents. *arXiv preprint arXiv:2506.02539*.

Ciyan Ouyang and Rui Hou. 2026. MemLineage: Lineage-Guided Enforcement for LLM Agent Memory. *Preprint*, arXiv:2605.14421.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. 2026. Formal policy enforcement for real-world agentic systems. *Preprint*, arXiv:2602.16708.

Haowen Pan, Pengyang Shao, Yunshan Ma, Yanzheng Jin, Naixin Zhai, Xiang Wang, and Xun Yang. 2026. MemForm: Benchmarking Memory Formation Poisoning in LLM Agents. *Preprint*, arXiv:2610.14431.

Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha S Bajwa, Vyas Raina, and Ivaxi Sheth. 2026a. PersistBench: When should long-term memories be forgotten by LLMs? *arXiv preprint arXiv:2602.01146*.

Sidharth Pulipaka, Stanislau Hlebik, Leonidas Raghav, Sahar Abdelnabi, Vyas Raina, Ivaxi Sheth, and Mario Fritz. 2026b. Hidden in memory: Sleeper memory poisoning in LLM agents. *Preprint*, arXiv:2605.15338v2.

Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. 2025. Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control. *arXiv preprint arXiv:2505.18279*.

Henry L Roediger III, Michelle L Meade, and Erik T Bergman. 2001. Social contagion of memory. *Psychonomic Bulletin & Review*, 8(2):365–371.

Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2025. Great, now write an article about that: The Crescendo multi-turn LLM jailbreak attack. In *USENIX Security Symposium*.

Ehsan Saleh, Roozbeh Nahavandi, Sama Salarian, Raef Bassily, and Andrew Perrault. 2026. AfterDeleteBench: Auditing post-deletion information persistence in LLM agents. *Preprint*, arXiv:2610.31366v1.

Daniel L. Schacter, Donna Rose Addis, and Randy L. Buckner. 2007. Remembering the past to imagine the future: The prospective brain. *Nature Reviews Neuroscience*, 8(9):657–661.

Daniel L Schacter, Kenneth A Norman, and Wilma Koutstaal. 1998. The cognitive neuroscience of constructive memory. *Annual Review of Psychology*, 49:289–318.

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703*.

Xiaonan Si, Meilin Zhu, Simeng Qin, Lijia Yu, Lijun Zhang, Shuaitong Liu, Xinfeng Li, Ranjie Duan, Yang Liu, and Xiaojun Jia. 2025. SeCon-RAG: A two-stage semantic filtering and conflict-free framework for trustworthy RAG. In *Advances in Neural*

Information Processing Systems 38, NeurIPS 2025, pages 78742–78771. Neural Information Processing Systems Foundation, Inc.

Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. 2025. PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows. In *2025 IEEE International Conference on eScience (eScience)*, pages 467–473. IEEE.

Larry R Squire, Lisa Genzel, John T Wixted, and Richard G Morris. 2015. Memory consolidation. *Cold Spring Harbor Perspectives in Biology*, 7(8):a021766.

Saksham Sahai Srivastava and Haoyu He. 2025. MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval. *arXiv preprint arXiv:2512.16962*.

Yongye Su, Wujiang Xu, Chaoji Zuo, and Elisa Bertino. 2026. ChronoMem: Version Control and Semantic Rollback for Large Language Model Agent Memory. *Preprint*, arXiv:2607.27773.

Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. 2024. Cognitive architectures for language agents (CoALA). *Transactions on Machine Learning Research*.

Fengzhou Sun, Yuan Zhang, Xintong Yu, and Jinyao Yan. 2026. EP-Mem: Elastic privacy memory for social relationship-aware LLM agents. *Preprint*, arXiv:2609.35233v1.

Elham Tabassi. 2023. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, National Institute of Standards and Technology, Gaithersburg, MD.

Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025a. MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents. In *Findings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2506.21605.

Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai. 2025b. RevPRAG: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis. In *Findings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ArXiv:2411.18948.

Yaxin Tang, Yijia Liu, Jiahe Lan, Zheng Yan, and Erol Gelenbe. 2026. Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey. *Information Fusion*, 127:103941.

Hanling Tian, Gengyu Zhang, Zeyang Sha, Jingying Wang, Yuhang Liu, Zhehao Huang, Kun Yang, and Xiaolin Huang. 2026. InjecMEM: Memory injection attack on LLM agent memory systems. *arXiv preprint arXiv:2608.23471*. Accepted at the 3rd Conference on Language Modeling (COLM 2026).

Bin Wang, Fan Wang, Pingping Wang, Jinyu Cong, Yang Yu, Yilong Yin, Zhongyi Han, and Benzheng Wei. 2026a. Agentic unlearning: When LLM agent meets machine unlearning. *arXiv preprint arXiv:2602.17692*.

Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. 2025. Unveiling privacy risks in LLM agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2502.13172.

Haoyu Wang, Christopher M Poskitt, and Jun Sun. 2026b. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE '26)*. ArXiv:2503.18666.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.

Shang Wang, Tianqing Zhu, Dayong Ye, and Wanlei Zhou. 2026c. When machine unlearning meets retrieval-augmented generation (RAG): Keep secret or forget knowledge? *IEEE Transactions on Dependable and Secure Computing*, 23(2):2030–2045.

Yiqi Wang, Caili Yu, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, and Taotao Cai. 2026d. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents. *Preprint*, arXiv:2608.10502v2.

Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. 2025. A-MemGuard: A proactive defense framework for LLM-based agent memory. *arXiv preprint arXiv:2510.02373*.

Yule Wen, Yanzhe Zhang, Jianxun Lian, Xiaoyuan Yi, Xing Xie, and Diyi Yang. 2026. Contextualized privacy defense for LLM agents. *arXiv preprint arXiv:2603.02983*.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025a. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*.

Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. 2025b. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. *arXiv preprint arXiv:2504.15965*.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025c. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*. ArXiv:2403.04960.

Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal. 2026. Certifiably robust RAG against retrieval corruption. In

IEEE Conference on Secure and Trustworthy Machine Learning (SaTML). ArXiv:2405.15556.

Bofan Xie, Chaokun Wang, Guanchen Ge, Songyao Wang, Fang Niu, Hang Zhang, and Wei Wang. 2026. Can agents forget when asked? a benchmark for memory unlearning in LLM agents. *Preprint*, arXiv:2610.18052v1.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*. NeurIPS 2025.

Zhenlin Xu, Xiaogang Zhu, Yu Yao, Minhui Xue, and Yiliao Song. 2026. From storage to steering: Memory control flow attacks on LLM agents. *arXiv preprint arXiv:2603.15125*.

Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. 2024. BadRAG: Identifying vulnerabilities in retrieval augmented generation of large language models. *Preprint*, arXiv:2406.00083.

Tiankai Yang, Jiate Li, Yi Nian, Shen Dong, Ruiyao Xu, Ryan Rossi, Kaize Ding, and Yue Zhao. 2026a. No attacker needed: Unintentional cross-user contamination in shared-state LLM agents. *arXiv preprint arXiv:2604.01350*.

Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. 2026b. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections. *arXiv preprint arXiv:2602.15654*.

Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, and Lei He. 2026. Forgetting without restarting: Execution-state unlearning for stateful LLM agents. *Preprint*, arXiv:2609.04875v1.

Sangyeon Yoon, Sunkyoung Kim, Hyesoo Hong, Wonje Jeung, Yongil Kim, Wooseok Seo, Heuiyeen Yeen, and Albert No. 2026. BenchPreS: A benchmark for context-aware personalized preference selectivity of persistent-memory LLMs. *arXiv preprint arXiv:2603.16557*.

Guangsheng Yu, Yanna Jiang, Qin Wang, Baihe Ma, and Xu Wang. 2026. K-Bench: A Benchmark for LLM Unlearning in Agentic Deployments. *Preprint*, arXiv:2609.12808.

Yosif Zaki and Denise J Cai. 2025. Memory engram stability and flexibility. *Neuropsychopharmacology*, 50:285–293.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*. ArXiv:2403.02691.

Baolei Zhang, Haoran Xin, Minghong Fang, Zhuqing Liu, Biao Yi, Tong Li, and Zheli Liu. 2025a. Traceback of poisoning attacks to retrieval-augmented generation. In *Proceedings of The Web Conference (WWW)*. ArXiv:2504.21668.

Baolei Zhang, Haoran Xin, Jiatong Li, Dongzhe Zhang, Minghong Fang, Zhuqing Liu, Lihai Nie, and Zheli Liu. 2025b. Benchmarking poisoning attacks against retrieval-augmented generation. *arXiv preprint arXiv:2505.18543*.

Cheng Zhang, Xuanning Cao, Zhicheng Zheng, and Hui Xiong. 2026a. MemCut: Every Secret Leaves a Trail in the Long-Term Memory of LLM Agents. *Preprint*, arXiv:2610.21405.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025c. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.02644.

Shuning Zhang, Lyumanshan Ye, Xin Yi, Jingyu Tang, Bo Shui, Haobin Xing, Pengfei Liu, and Hewu Li. 2024. "Ghost of the past": identifying and resolving privacy leakage from LLM's memory through proactive user interaction. *Preprint*, arXiv:2410.14931.

Xuanye Zhang, Yongsen Zheng, Zhuqin Xu, Kaiyu Zhou, Bowen Shen, Haoran Ou, Tianwei Zhang, and Kwok-Yan Lam. 2026b. MemMorph: Tool hijacking in LLM agents via memory poisoning. *Preprint*, arXiv:2605.26154v1.

Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025d. A survey on the memory mechanism of large language model-based agents. *ACM Trans. Inf. Syst.*, 43(6).

Zeyu Zhang, Ziliang Guo, Yihang Sun, Xichong Zhang, Xixuan Hao, Zehao Lin, Yang Zhang, Xiaoyan Zhao, Tong Shen, Bo Tang, Zhi-Qin John Xu, Junchi Yan, Haofen Wang, Xu Chen, Feiyu Xiong, Zhiyu Li, and Tat-Seng Chua. 2026c. Metis: Memory foundation model. *arXiv preprint arXiv:2607.26760*.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Zhenhao Li, Yue Chen, Huaxiu Yao, Hamed Haddadi, and Emine Yilmaz. 2026. TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation. In *AAAI 2026 Workshop on Trust and Control in Agentic AI (TrustAgent)*, Singapore.

Wei Zou, Mingwen Dong, Miguel Romero Calvo, Shuaichen Chang, Jiang Guo, Dongkyu Lee, Xing Niu, Xiaofei Ma, Yanjun Qi, and Jiarong Jiang. 2026. Poison once, exploit forever: Environment-injected memory poisoning attacks on web agents. *arXiv preprint arXiv:2604.02623*.

Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *USENIX Security Symposium*. ArXiv:2402.07867.

Appendix

A Architectural Governance Primitives

Table 3 summarizes the five VMG primitives from Section 5, specifying for each the representative threat, corresponding defense direction, and proposed evaluation metric.

Use-time endorsement and origin-bound action authority complement admission and provenance controls (Jiang et al., 2026; Louck, 2026). Selective dependency repair complements historical restoration but has a different recovery target (Wang et al., 2026d).

B Human-Memory Bridge / Conceptual Grounding

Our lifecycle memory view is grounded in a limited analogy with human memory. Cognitive neuroscience frames memory not as a fixed archive but as a reconstructive and reconsolidatable system where experiences are encoded, consolidated, reactivated, and sometimes rewritten as context and goals evolve (Squire et al., 2015; Schacter et al., 1998, 2007; Nader et al., 2000; Zaki and Cai, 2025). This perspective underscores why long-term memory in LLM agents cannot be reduced to mere context extension or passive storage (Zhang et al., 2026c). Profile memories, episodic logs, summaries, and retrieval stores determine what persists, what is later reactivated, how traces are compressed, and how the agent interprets its own past. The security question is therefore not only whether a single input is malicious, but whether an untrusted trace can be written, consolidated, retrieved, elevated, propagated, or deleted across the memory lifecycle.

The analogy also clarifies the primitives needed for Verifiable Memory Governance (VMG). In human memory, misinformation often operates through source-monitoring failure, confidence inflation, reconsolidation after recall, and social contagion rather than direct overwrite (Loftus, 2005; Roediger III et al., 2001; Nader et al., 2000). The corresponding agent-level failures include provenance failure, read-time rewriting through summarization or reflection, and cross-agent contamination. Injected content may be misattributed as the agent's own experience (Dong et al., 2025; Tian et al., 2026), promoted through compression into more authoritative lesson memory (Packer et al., 2023; Xu et al., 2025), or propagated via shared memory and inter-agent communication (Gu et al., 2024; Men et al., 2025). Robust LTM security therefore needs to operate across the full lifecycle. Write authorization and provenance visibility protect the origin of memory, principal-scoped retrieval limits who can reactivate it, and rollbackability together with verified forgetting determines whether corrupted or sensitive traces can be removed after they have spread.

C Methodology for Paper Selection

We conduct a structured literature review that combines literature retrieval with citation-based expansion. The original review searched academic databases and repositories, including the ACL Anthology, ACM Digital Library, IEEE Xplore, USENIX proceedings, OpenReview, and arXiv. Its keyword categories covered memory poisoning / injection, long-term / episodic / agent memory, RAG poisoning, memory extraction, multi-agent memory, memory governance, forgetting, and rollback. The retained search documentation covers databases and keyword categories, but not exact queries, a search cutoff, or item-level screening decisions.

We further expand the candidate set by tracing relevant references and citations through Google Scholar, arXiv, and Semantic Scholar. In total, we screened more than 200 candidate works, among which approximately 90 were included in the survey. These counts describe the original review.

The supplementary review, completed on 23 October 2026, added six memory-security and evaluation papers and two works on memory-native representations and decision value. Selection focused on distinct mechanisms and evidence relevant to existing lifecycle stages. We checked arXiv records and the passages supporting each addition.

A second targeted update, completed on 30 October 2026, adds 21 works to the published v4 baseline. Candidates were selected for distinct attack mechanisms, governance interfaces, recovery targets, privacy risks, or complementary survey perspectives. We verified official bibliographic records and the full-text passages supporting the added claims, using fixed arXiv versions.

Table 3: Architectural governance primitives.

Primitive	Representative Threat	Control Direction	Proposed Metric
Write Authorization (WA)	Query-induced injection; environment-injected poisoning (Dong et al., 2025; Tian et al., 2026; Zou et al., 2026)	Typed write validation; human or policy arbitration (Nguyen et al., 2025; Jacob et al., 2025; Palumbo et al., 2026)	Adaptive injection-survival rate at bounded false-positive rate (FPR) (Nasr et al., 2026)
Provenance Visibility (PV)	Source-monitoring failure; imitation-based grafting (Srivastava and He, 2025)	Provenance-gated retrieval; lineage-preserving compression (Moreau and Missier, 2013; Souza et al., 2025)	Fraction of entries traceable to source event
Principal-Scoped Retrieval (PS)	Cross-user contamination; black-box extraction (Yang et al., 2026a; Wang et al., 2025; El Yagoubi et al., 2026; He et al., 2026)	Capability- or label-based retrieval policy (Li et al., 2025a; Costa et al., 2025; Mireshghallah et al., 2026); ownership enforcement (Mudgal et al., 2026)	Cross-principal leakage rate under scripted query suites
Rollbackability (RB)	Compression-amplified toxins; behavioral drift (Packer et al., 2023; Xu et al., 2025)	Versioned restoration; forensic localization (Su et al., 2026; Zhang et al., 2025a)	Restoration accuracy under current policies; recovery coverage and latency
Verified Forgetting (VF)	Residual derivatives; reappearance in summaries (Zhang et al., 2024; Pulipaka et al., 2026a)	Deletion coordinated over declared substrates and derivatives (Bourtoule et al., 2021; Maini et al., 2024; Wang et al., 2026c,a)	Post-deletion exposure under declared access and probe conditions (Maini et al., 2024; Wang et al., 2026c; Zhang et al., 2026a; Saleh et al., 2026)

D VMG Operationalization

Equations (1)–(6) specify the VMG criteria. Operational checks must declare the system boundary, policies, and evaluation scope. Possible metrics are as follows:

- WA: compare authenticated write events and their recorded decisions against the declared policy. Use published injection attacks (e.g., MINJA, eTAMP) to measure injection survival and false positives.
- PV: under designated audit credentials, resolve provenance for each entry version and compare it with controlled or instrumented dependency traces. Test lower-privilege queries for prohibited metadata disclosure. Include summarization, merging, and tool echoes; separately test whether transformations improperly increase action authority (Louck, 2026).
- PS: issue entry-returning retrieval queries under different principal identities and measure results exceeding the requester's authorized scope. Test release of synthesized or aggregated outputs separately under the applicable disclosure policy.
- RB: declare the live-memory projection and recovery points, trigger rollback, and compare the restored view with $P_t(L(M_{t_0}))$, including current erasure and revocation constraints. Check reconstruction of required indices and caches, and preservation of permitted audit history. Report restoration accuracy, coverage, and latency separately.
- VF: use tiered probes (direct, paraphrased, associative, multi-turn) to test reconstruction under declared access paths and judge criteria, reporting targets, accessible derivatives, probe counts, and uncertainty. Report reconstruction, reinsertion, later-session recall, preservation, and relearning separately (Saleh et al., 2026; Xie et al., 2026); declare whether tool reacquisition is permitted and distinguish it from surviving memory (Lin et al., 2026a). K-Bench extends observation across agent channels in separate weight, prompt, and retrieval settings (Yu et al., 2026).

Paired deletion audits. AfterDeleteBench pairs an exposed-then-deleted state with a matched never-exposed control (Saleh et al., 2026). Their exposure difference estimates the incremental effect of prior exposure; it does not replace the absolute bound in VF, since both states can have high disclosure.

E Real-System Analysis

We map mechanisms described in Mem0 (arXiv: 2504.19413v1, 28 April 2025) (Chhikara et al., 2025) and MemOS (arXiv:2507.03724v1, 4 July 2025) (Li et al., 2025b) to the VMG requirements.

The comparison uses these fixed paper versions, checked on 23 October 2026.

- Mem0 describes LLM-selected ADD, UPDATE, DELETE, and NOOP operations (Section 2.1). Its graph variant, Mem0^g, records creation timestamps and retains invalidated relations for temporal reasoning (Section 2.2). These support tracking memory changes; the cited version leaves dedicated write-authorization and rollback protocols unspecified.

- MemOS describes provenance and version metadata, read/write/share scopes, snapshots, historical rollback, and lifecycle policies (Sections 4.2, 5.3.2, 5.4.3, and 5.5). These provide implementation hooks for VMG. Evaluating them requires testing lineage completeness across transformations, authorization under adversarial updates, and erasure of derivatives and propagated copies.

Bemula complements these mechanism descriptions with measurements tied to the tested implementation, model, and request protocol (Xie et al., 2026).

Author relationships. MemPrivacy includes survey coauthors Feiyu Xiong and Zhiyu Li. MemOS, MemOps, Metis, and the memory-value paper also share coauthors with this survey, while MemT-1.0 is attributed to the MemTensor Team, with which several survey authors are affiliated.

F Revision Notes (30 October 2026)

The 30 October 2026 update adds 21 references and defines the four security objectives. Equation (4) now projects onto the live-memory recovery view, so its equality does not rewind append-only audit history.

G Selected Evidence Within the Lifecycle

Table 4 maps the 19 technical additions to the existing taxonomy; the two added surveys appear in Table 1. Phases locate the mechanisms, and primary objectives identify their main risks or tradeoffs. The final column records relevant policy boundaries and evaluation conditions.

Table 4: The 19 technical works added in the 30 October update mapped to the existing lifecycle and their primary security objectives. I: integrity; C: confidentiality; A: availability; G: governance. Objectives may overlap.

Work or mechanism	Lifecycle location	Primary objectives	Evidence and interpretation
Hidden in Memory; MPBench (Pulipaka et al., 2026b; Dash et al., 2026)	Write–Store–Execute	I, G	Persistent instructions and skills connect admission to later behavior. Writing success, conditional activation, and end-to-end harm have different denominators.
ShadowMerge; MemMorph (Luo et al., 2026b; Zhang et al., 2026b)	Store–Retrieve–Execute	I	Graph merging and memory rewriting can preserve or amplify adversarial influence. The studies use different write-access and optimizer-knowledge assumptions.
Self-propagating misalignment (Das et al., 2026)	Write–Share–Execute	I, G	Prompt-induced misaligned agents transmit goals through persistent state, including files.
ADAM; MRMMIA (Lyu et al., 2026; Chen et al., 2026a)	Retrieve–Execute–Share	C	Adaptive extraction and known-candidate membership are distinct endpoints. Downstream disclosure is evaluated under specified access privileges, query budgets, and repeated probes.
MemLeak (Mudgal et al., 2026)	Retrieve–Execute–Share	C, G	Ownership enforcement restricts which users' entries reach context. Correct partitioning or metadata filtering also blocks the studied path.
EP-Mem (Sun et al., 2026)	Store–Retrieve–Execute–Share	C, G	User-configured audience and relationship policies govern disclosure across memory operations. Appropriate release after retrieval is a separate boundary from ownership-based access.
MemPrivacy (Chen et al., 2026d)	Store–Retrieve–Execute	C	Local substitution with typed placeholders and a local restoration map reduce sensitive data exposed to cloud-side processing.
Disclosure Begets Disclosure (Kaneko et al., 2026)	Execute–Write–Retrieve	C	In multi-session simulations, memory-conditioned responses elicit replies that reinforce personal attributes. Response-side intervention addresses an unintended disclosure loop without requiring an attacker.
Utility Under Attack (Karunanidhi, 2026)	Write–Retrieve–Execute	I, A	Plain false assertions evade the evaluated screening. Strong provenance penalties can improve accuracy but also suppress legitimate untrusted evidence under the measured similarity conditions.
TMA-NM; PerMIT (Louck, 2026; Jiang et al., 2026)	Store–Execute–Share	I, G	Authenticated origins and explicit endorsement constrain action authority beyond memory admission. Bounded TLA+ checks and Lean-checked single-user integrity have different scopes.

Continued on next page

Table 4: Additional technical evidence within the lifecycle (continued).

Work or mechanism	Lifecycle location	Primary objectives	Evidence and interpretation
Dependency-guided repair (Wang et al., 2026d)	Execute–Forget & Rollback	I, A	Dependency-based selective repair preserves independently supported state and recomputes affected steps. Its recovery target differs from historical restoration; report recurrence and retained utility.
Execution-state unlearning (Yao et al., 2026)	Execute–Forget & Rollback	C	Clean-prefix restoration and sanitized replay target a counterfactual execution state. The result depends on reconstructible runtime state, fixed randomness or determinism, and compatible replay observations.
AfterDeleteBench; Bemula (Saleh et al., 2026; Xie et al., 2026)	Forget–Retrieve–Write	C, A	Paired audits and natural-language requests distinguish removal from reconstruction, preservation, reinsertion, and relearning.
AUBench (Lin et al., 2026a)	Forget–Retrieve–Execute	C, G	Parameter, memory, and tool-access channels motivate explicit audit boundaries. Reacquisition through a permitted tool must be distinguished from surviving memory.